**VNRVJIET** **Name of the Experiment:** ____________________

**Name of the Laboratory:** ____________________________ **Experiment No:**________ **Date:** ________________

**Experiment 1**

**Installing Cloudera QuickStart VM**

Cloudera provides virtual machine (VM) images of complete Apache Hadoop clusters, making it quick and easy to get started with Cloudera CDH.

**Note:** These are free for personal use, but do require you to register your details on the Cloudera website prior to download. Remember to check your system meets the minimum requirements.

**I'm familiar with running VMs...**

Go ahead and download the appropriate Cloudera Quickstart VM if you are already familiar with running virtual machines, and have VM software installed. Clone the example files from the FLbigdataD2D GitHub repository once you have Cloudera QuickStart up and running.

**I do not have VM software installed...**

Follow the instructions to first install VirtualBox (virtual machine client) and then get started with the Cloudera Quickstart VM.

**Note:** The Cloudera Quickstart VM download file is approx. 5GB, so download time will vary depending on your connection speed. There are online calculators you can use if you'd like an estimate of how long the download may take. Conduct a web search for: how long will my download take.

**1. Install Virtual Box (virtual machine client)**

**a) Download VirtualBox.**

Ensure you download the correct version of VirtualBox for your operating system.

PC users: VirtualBox 5.0.14 for Windows hosts (112MB)

MAC users: VirtualBox 5.0.14 for OS X hosts (86.2MB)

VNRVJIET Name of the Experiment: ____________

Name of the Laboratory: ____________ Experiment No:_______ Date: ____________

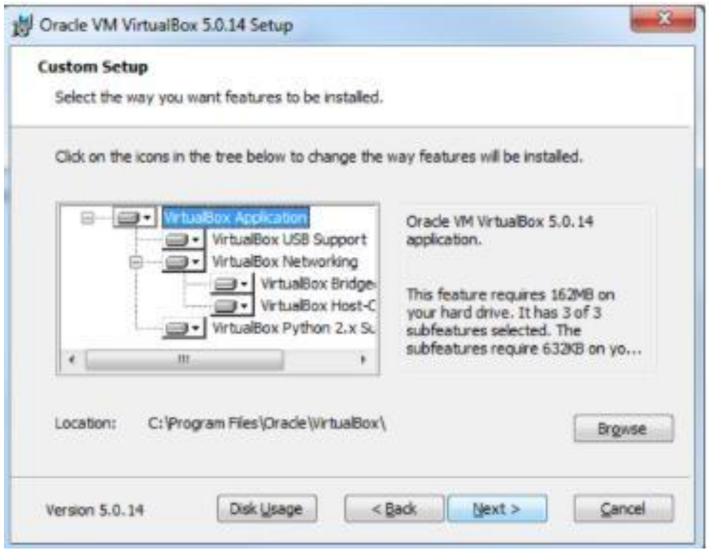
b) Install VirtualBox.

MAC users: Double click the .dmg file you have downloaded and follow the instructions.

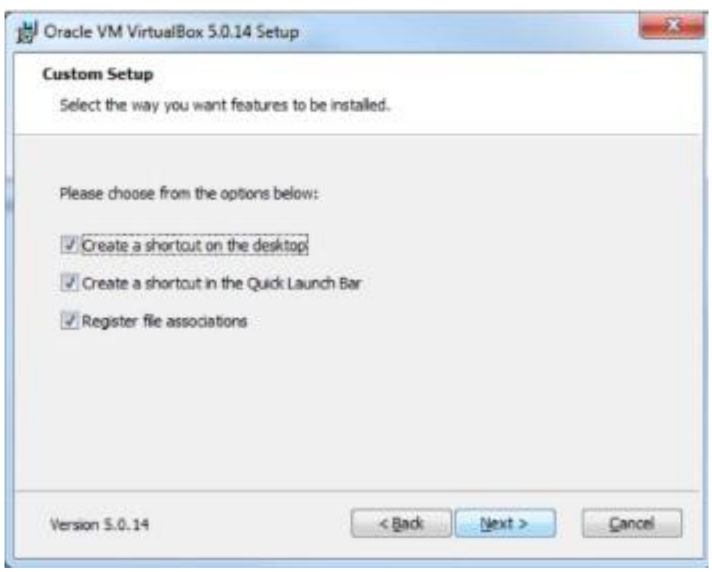

**PC users:** Double click the .exe file you have downloaded and follow the instructions.

i. Double click .exe file and click Run.

ii. Click Next accepting the suggested custom setup to install features.



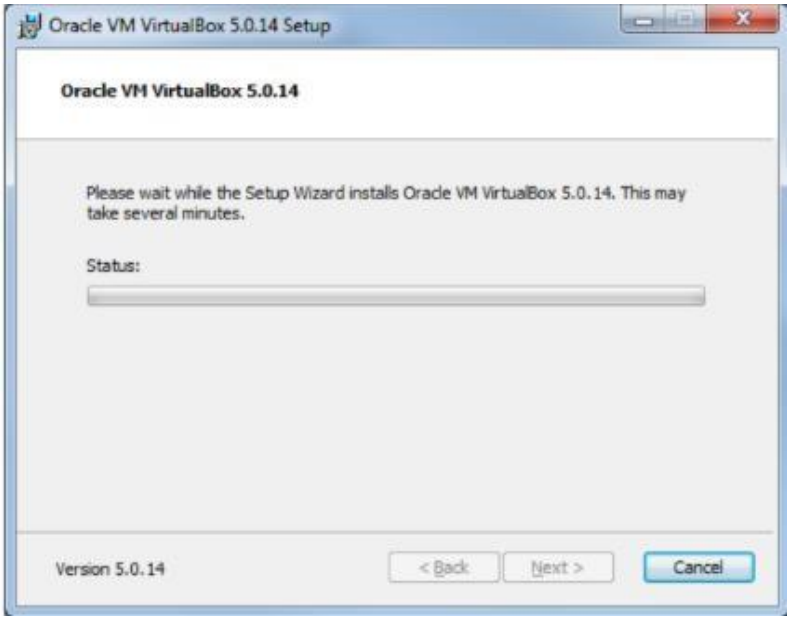

iii. Click Next accepting the suggested custom setup.




iv. Click Yes.

v. Click Install.

vi. Wait for the install to complete. You may receive prompts to confirm the installation as it progresses. Click Yes for each prompt to continue the installation




vii. Click Finish when the installation is complete.

**2. Download ClouderaCDH VM for VirtualBox.**

a) Select the virtual machine client you are using from the drop down list (VirtualBox).

b) Click Download now to begin downloading the installation zip archive file for Cloudera QuickStart VM.

c) Register and sign in or complete the product interest form.

Before you can download the file you must complete this step. You are required to complete all fields before you can continue. Cloudera may use your details to contact you.


**3. Extract files from the Cloudera Quickstart VM zip archive.**

Ensure you have sufficient space to extract the archive if you encounter problems.

Alternatively try using 7-zip (Windows) or TheUnArchiver (Mac) which work well with large files.

4. Open the Cloudera Quickstart VM in VirtualBox.

a) Launch VirtualBox.

b) Click on File - Import appliance and select the .ovf file from the location where you have saved the extracted Cloudera Quickstart VM files. Click Next.

Leave 'Reinitialise the MAC address of all network cards' unticked.

c) Click Import.

5. Start your Cloudera Quickstart VM instance.

a) Select cloudera-quickstart-vm from the left panel.

b) Click the Start (green arrow) icon.


**Note:** The VM can take some time to start up. You'll see the blue Cloudera desktop and the Welcome to Cloudera Quickstart VM page once it's ready.

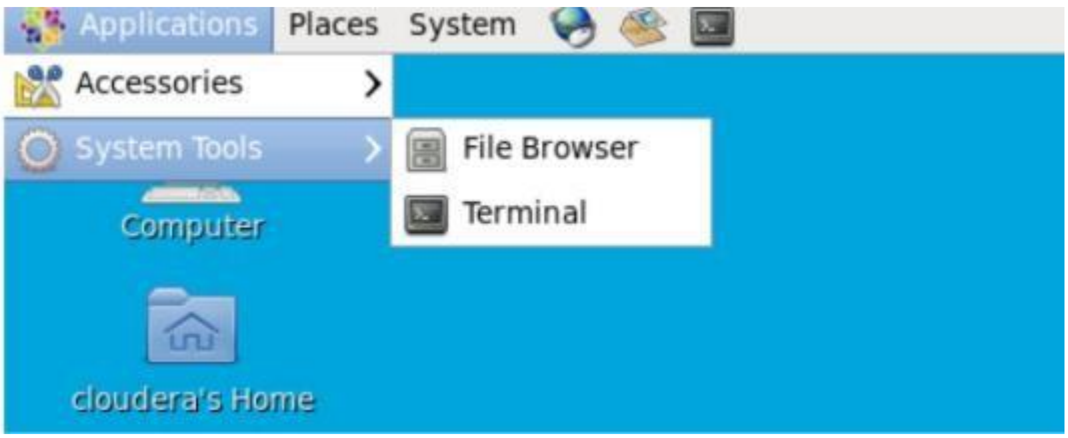

**Tip:** Once you are finished using Cloudera, remember to shut down the VM by choosing

System - Shut down.

**6. Clone the example files from the FLbigdataD2D GitHub repository.**

a) Open the Terminal.

Select Applications > System Tools > Terminal.

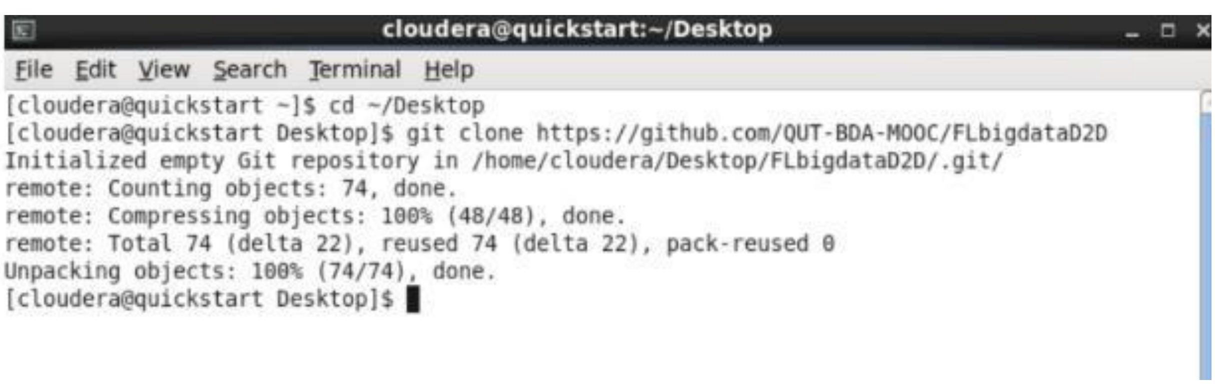



b) Change to the Desktop folder.

Type cd ~/Desktop in the Terminal window.

c) Clone the example files from GitHub.

Type git clone https://github.com/QUT-BDA-MOOC/FLbigdataD2D in the Terminal window.



# VNRVJIET

**Name of the Experiment:** ______________

**Name of the Laboratory:** ______________

**Experiment No:**______ **Date:** ______________

**LINUX COMMANDS**

These are the most commonly used commands for basic Linux file and directory management.

**Directory Management**

1. Create a new directory → mkdir
2. Create a directory named project → mkdir project
3. Create multiple directories at once → mkdir dir1 dir2 dir3
4. Create a directory inside another directory → mkdir parent/child

**Removing Directories**

5. Remove an empty directory → rmdir dirname
6. Delete a directory along with its contents → rm -r dirname

**Navigation & Listing**

7. Display the current directory → pwd
8. List files and directories → ls
9. List all files including hidden files → ls -a

**File Creation & Viewing**

10. Create an empty file → touch filename
11. Create a file named data.txt → touch data.txt
12. View the contents of a file → cat filename
13. Write text into a file using a command → echo "text" > filename

**File Operations**

14. Copy a file → cp source destination
15. Rename a file → mv oldname newname

16. Delete a file → rm filename

17. Move a file from one directory to another → mv filename

/path/to/directory

**Permissions & Size**

18. Check file permissions → ls -l filename

19. Change file permissions → chmod permissions filename

**Example:** chmod 755 filename

20. Show the size of a file → ls -lh filename

**Basic UNIX Commands to Create a Directory And File**

1. pwd //Present Working Directory

2. whoami //which user account is currently active

3. mkdir <directory name> //to create a directory

4. ls //to check list of files and directories

5. cd firstdir //change directory

6. cd .. //change to parent directory

7. touch <file name> //creates an empty file

8. vi <filename> //to view,edit a file

Common vi command in linux

- i → insert text
- Esc → return to command mode
- :w → save file
- :q → quit
- :wq → save and quit
- :q! → quit without saving

9. cat <filename> //display the content of a fil

1. HDFS (Storage)

A. Hadoop Storage File system: Your first objective is to create a directory structure in HDFS using HDFS commands. Create the local files using Linux commands and move the files to HDFS directory and vice versa.

I. Write a command to create the directory structure in HDFS.

II. Write a Command to move file from local unix/linux machine to HDFS.

B. Viewing Data Contents, Files and Directory Try to perform these simple steps:

Write HDFS command to see the contents of files in HDFS.

C. Getting Files data from the HDFS to Local Disk

I. Write a HDFS command to copy the file from HDFS to local file system. To process any data first move the data in HDFS. All files stored in HDFS can be accessed using HDFS commands.

Ans) A)

I) [cloudera@quickstart ~]$ hadoop fs -mkdir Directory

II) [cloudera@quickstart ~]$ hadoop fs -put /home/cloudera/ Desktop/myfile.txt Directory

[cloudera@quickstart ~]$ hadoop fs -ls

Directory O/P:-

Found 1 items
-rw-r--r-- 1 cloudera cloudera 14 2023-05-02 01:36 Directory/myfile.txt

B) [cloudera@quickstart ~]$ hadoop fs -cat Directory/myfile.txt Hello, World!

C) [cloudera@quickstart ~]$ hadoop fs -get Directory/myfile.txt /home/cloudera/Desktop/myfile1.txt

# VNRVJIET

**Name of the Experiment:** ____________________

**Name of the Laboratory:** ____________________

**Experiment No:**_______ **Date:** ____________

**Experiment 2**

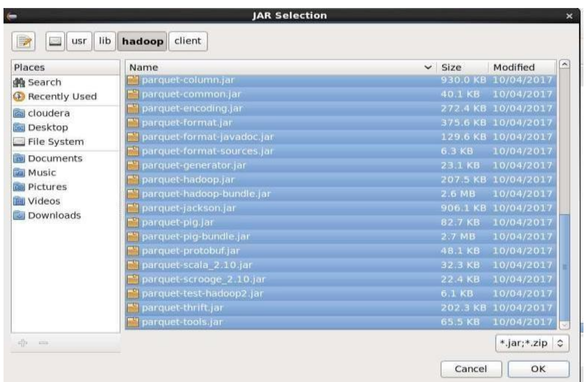
A. Develop MapReduce example program in a MapReduce environment to find out the number of occurrences of each word in a text file

1 .Create a new java project .Give name as WordCount.

Then click next

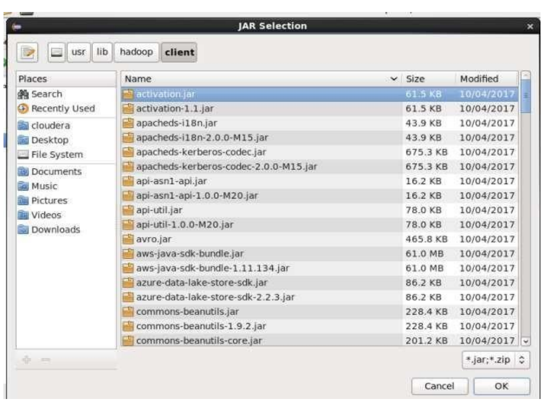
2. then go to libraries there click on add external jar files then File system->usr->lib->hadoop

Copy all the jar files




Then click on ok.

3. Again click on add external jar files, goto File system->usr->lib->hadoop->client



Select all the files and then click ok.

4. click on finish

5.Now click on wordCount->src create a new class WordCount.java

From apache documentation of hadoop goto MapReduce tutorials copy the java code

Java code:

```
import
java.io.IOException;
import
java.util.StringTokenizer;

import
org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import
org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import
org.apache.hadoop.mapreduce.Job;
import
org.apache.hadoop.mapreduce.Mapper;
import
org.apache.hadoop.mapreduce.Reducer;
import
org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import
org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {

  public static class TokenizerMapper
       extends Mapper<Object, Text, Text, IntWritable>{
```

```
private final static IntWritable one = new
IntWritable(1); private Text word = new Text();

public void map(Object key, Text value, Context context
                ) throws IOException, InterruptedException {
  StringTokenizer itr = new
  StringTokenizer(value.toString()); while
  (itr.hasMoreTokens()) {
    word.set(itr.nextToke
    n());
    context.write(word,
    one);
  }
}
}

public static class IntSumReducer
     extends
  Reducer<Text,IntWritable,Text,IntWritable> {
  private IntWritable result = new IntWritable();

  public void reduce(Text key, Iterable<IntWritable>
                     values, Context context
                     ) throws IOException,
    InterruptedException { int sum = 0;
    for (IntWritable val :
      values) { sum +=
      val.get();
    }
    result.set(sum);
    context.write(key,
    result);
  }
}
```

```
  public static void main(String[] args) throws Exception {
    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "word count");
    job.setJarByClass(WordCount.class);
    job.setMapperClass(TokenizerMapper.class);
    job.setCombinerClass(IntSumReducer.class);
    job.setReducerClass(IntSumReducer.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
  }

}
```

6. save the java code and run it.

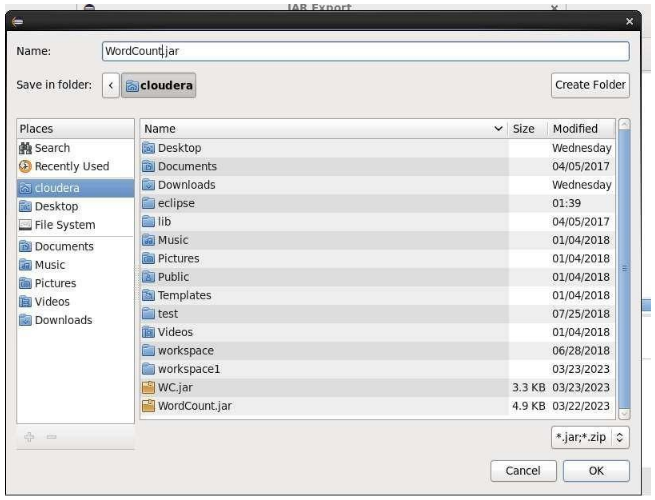
7. Right click on WordCount folder->export->select jar file->click on next


8. Name the jar file and select the required location the jar file to be exported -
>ok->finish

**VNRVJIET** **Name of the Experiment:** ______________

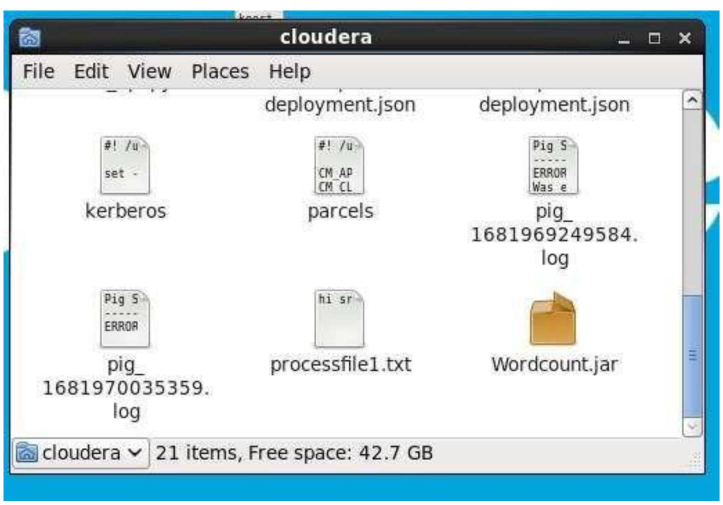
**Name of the Laboratory:** ______________

**Experiment No:**______ **Date:** ______________





1. Go to the location where we exported the jar file and verify it.




2. Now open terminal and do the following operations

```
[cloudera@quickstart ~]$ pwd
/home/cloudera
[cloudera@quickstart ~]$ ls
cloudera-manager  eclipse                     Music     Videos
cm_api.py         enterprise-deployment.json  parcels   Wordcount.jar
Desktop           express-deployment.json     Pictures  workspace
Documents         kerberos                    Public
Downloads         lib                         Templates
```

Check for wordcount.jar file

```
[cloudera@quickstart ~]$ cat >/home/cloudera/processfile1.txt
hi sravanthi hello keerthana
```

```
[cloudera@quickstart ~]$ hdfs dfs -mkdir /input
[cloudera@quickstart ~]$ hadoop dfs -put /home/cloudera/Desktop/processfile.txt
/input/processfile1.txt
```

```
[cloudera@quickstart ~]$ hdfs dfs -cat /input/processfile1.txt
hello sravanthi
keerthana hi hi
hi sravanthi
hello
```

```
[cloudera@quickstart ~]$ hadoop jar /home/cloudera/Wordcount.jar WordCount
/input/processfile1.txt /out
```

```
[cloudera@quickstart ~]$ hdfs dfs -ls /out
Found 2 items
-rw-r--r--   1 cloudera supergroup          0 2023-04-12 22:05 /out/_SUCCESS
-rw-r--r--   1 cloudera supergroup         37 2023-04-12 22:05 /out/part-r-00000
```

Check the number /out/part-r-00000

```
[cloudera@quickstart ~]$ hdfs dfs -cat /out/part-r-00000
```

The final output will be

```
hello   2
hi      3
keerthana       1
sravanthi       2
```

**Experiment 3**

3. Data Processing Tool – Hive (NOSQL query based language) Hive command line tool allows you to submit jobs via bash scripts.

Identifying properties of a data set:

We have a table 'user data' that contains the following

fields: data_date: string

user_id:

string

properties

: string

The properties field is formatted as a series of attribute=value

pairs. Ex: Age=21; state=CA; gender=M;

Ans)

I. Create the table in HIVE using hive nosql based query.

```
hive> CREATE TABLE IF NOT EXISTS user_data1 ( data_date
string, user_id string, properties Map<string,string> )
> ROW FORMAT DELIMITED
> FIELDS TERMINATED BY ','
> COLLECTION ITEMS TERMINATED BY '#'
> MAP KEYS TERMINATED BY '@'
> STORED AS TEXTFILE;

OK
Time taken: 0.091 seconds
```

II. Fill the table with sample data by using some sample data bases.

Create a text file with dummy data and load it using the following command.

Dummy Data:

22-04-2023,1,Age@21#State@CA#Gender@M
23-04-2023,2,Age@21#State@NY#Gender@F
24-04-2023,3,Age@22#State@OH#Gender@F
25-04-2023,4,Age@22#State@OH#Gender@M
26-04-2023,5,Age@23#State@TX#Gender@F
27-04-2023,6,Age@23#State@TN#Gender@M

hive> LOAD DATA INPATH 'Directory/user_database.txt' overwrite into table user_data1;

Loading data to table
default.user_data1 chgrp: changing
ownership of
'hdfs://quickstart.cloudera:8020/user/hive/warehouse/user_data1/user_d
atab ase.txt': User does not belong to supergroup
Table default.user_data1 stats: [numFiles=1, numRows=0,
totalSize=234, rawDataSize=0]
OK
Time taken: 0.699 seconds

hive> select * from
user_data1; OK
22-04-2023 1
{"Age":"21","State":"CA","Gender":"M","":null}
23-04-2023 2
{"Age":"21","State":"NY","Gender":"F","":null}
24-04-2023 3
{"Age":"22","State":"OH","Gender":"F","":null}
25-04-2023 4
{"Age":"22","State":"OH","Gender":"M","":null}
26-04-2023 5
{"Age":"23","State":"TX","Gender":"F","":null}
27-04-2023 6
{"Age":"23","State":"TN","Gender":"M","":null}
Time taken: 0.394 seconds, Fetched: 6 row(s)

III. Write a program that produces a list of properties with minimum value(min_value), largest value(max_value) and number of unique values. Before you start, execute the prepare step to load the data into HDFS.

IV.

Minimum values:

```
hive> select t.my_key, min(t.my_key_value) as min_value
    > from (
    > select explode(properties) as (my_key,my_key_value)
    > from user_data1
    > ) t
    > group by t.my_key;
...

Age         21
Gender      F
State       CA
Time taken: 26.099 seconds, Fetched: 3 row(s)
```

Maximum values:

```
hive> select t.my_key, max(t.my_key_value) as max_value
    > from (
    > select explode(properties) as (my_key,my_key_value)
    > from user_data1
    > ) t
    > group by t.my_key;
...

OK
Age         23
Gender      M
State       TX
Time taken: 22.254 seconds, Fetched: 3 row(s)
```

Unique Coun

```
hive> select tmy_key, count(distinct t.my_key_value) as unique_count
    > from (
    > select explode(properties) as (my_key,my_key_value)
```

```
   > from user_data1
   > ) t
   > group by t.my_key;
...

Age         3
Gender      2
State       5
Time taken: 19.956 seconds, Fetched: 3 row(s)
```

V. Generate a count per state.

```
hive> select my_value, count(*) as state_count
   > from (
   > select explode(properties) as (my_state, my_value)
   > from user_data1
   > ) t
   > where t.my_state = 'State'
   > group by t.my_value;
...

CA          1
NY          1
OH          2
TN          1
TX          1
Time taken: 21.772 seconds, Fetched: 5 row(s)
```

**Experiment 4**

Data Processing Tool – Pig (Latin based scripting lang)

Pig command line tool like the Hive allows you to submit jobs via bash scripts.

A) Simple Logs
We have a set of log files and need to create a job that runs every hour and perform some calculations.
The log files are delimited by a 'tab' character and have the following fields:

a) site
b) hour_of_day
c) page_views
d) data_date

The log files are located on the prepare folder. Load them in HDFS at data/ pig/ simple_logs folder and use them as the input.
Important: In order to load tab delimited files use pigStorage ('\u0001').

Lab Instructions:
Create a program to:

I. Calculate the total views per hour per day.
II. Calculate the total views per day.
III. Calculate the total counts of each hour across all days.
IV. We can write word count script by passing text file as input

Ans) Create a text file with dummy data.

Dummy data:

```
"site1",1,10,"2023-04-27"
"site2",1,5,"2023-04-27"
"site1",2,15,"2023-04-27"
"site2",2,20,"2023-04-27"
"site1",1,5,"2023-04-28"
"site2",1,10,"2023-04-28"
"site1",2,25,"2023-04-28"
"site2",2,30,"2023-04-28"
```

i, ii & iii)

Create a pig script file with following commands and execute it.

```
-- Load the log data from HDFS

logs = LOAD '/home/cloudera/sample_logs.txt' USING PigStorage(',') AS
(site:chararray, hour:int, views:int, date:chararray);

-- Calculate total views per hour per day

hourly_views = GROUP logs BY (date, hour);

hourly_views_sum = FOREACH hourly_views GENERATE group.date
AS date, group.hour AS hour, SUM(logs.views) AS views;

-- Calculate total views per day

daily_views = GROUP logs BY

date;

daily_views_sum = FOREACH daily_views GENERATE group AS date,
SUM(logs.views) AS views;

-- Calculate the total counts of each hour across all days

hour_counts = GROUP logs BY hour;

hour_counts_sum = FOREACH hour_counts GENERATE group AS hour,
SUM(logs.views) AS views;

-- Store the results in HDFS

STORE hourly_views_sum INTO '/home/cloudera/hourly_views' USING
PigStorage(',');
STORE daily_views_sum INTO '/home/cloudera/daily_views' USING
PigStorage(',');

STORE hour_counts_sum INTO '/home/cloudera/hour_counts' USING
PigStorage(',');
```

O/P:- It is stored at the given path, i.e., /home/cloudera. The output is produced as a folder containing a text file. The text file has the expected answer.

1) "2023-04-27",1,15

"2023-04-27",2,35

"2023-04-28",1,15

"2023-04-28",2,55

2)1
,30

2,90
3)"2023-04-27",50

"2023-04-28",70
4)Create a pig script file with following commands and execute it.
Give some text file's path as input instead of /home/cloudera/myfile.txt.

```
data = LOAD '/home/cloudera/myfile.txt' AS (line:chararray);
words = FOREACH data GENERATE FLATTEN(TOKENIZE(line)) AS
word;
word_counts = GROUP words BY word;
word_count_totals = FOREACH word_counts GENERATE group AS word,

COUNT(words) AS count;
STORE word_count_totals INTO '/home/cloudera';
```

O/P:- It is stored at the given path, i.e., /home/cloudera. The output is produced as a folder containing a text file. The text file has the expected answer.

Hello 1
World! 1

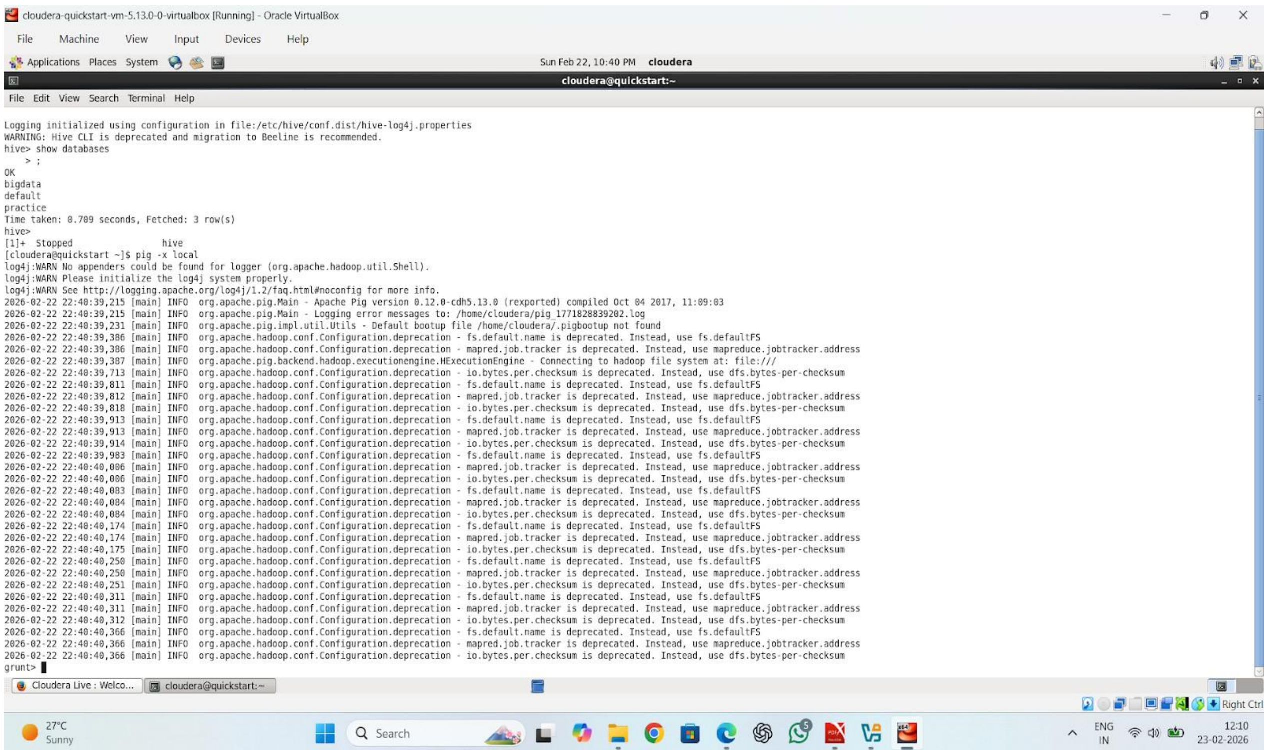
**VNRVJIET** **Name of the Experiment:** ____________

**Name of the Laboratory:** ____________

**Experiment No:**______ **Date:** ____________

-- to get grunt cell command is

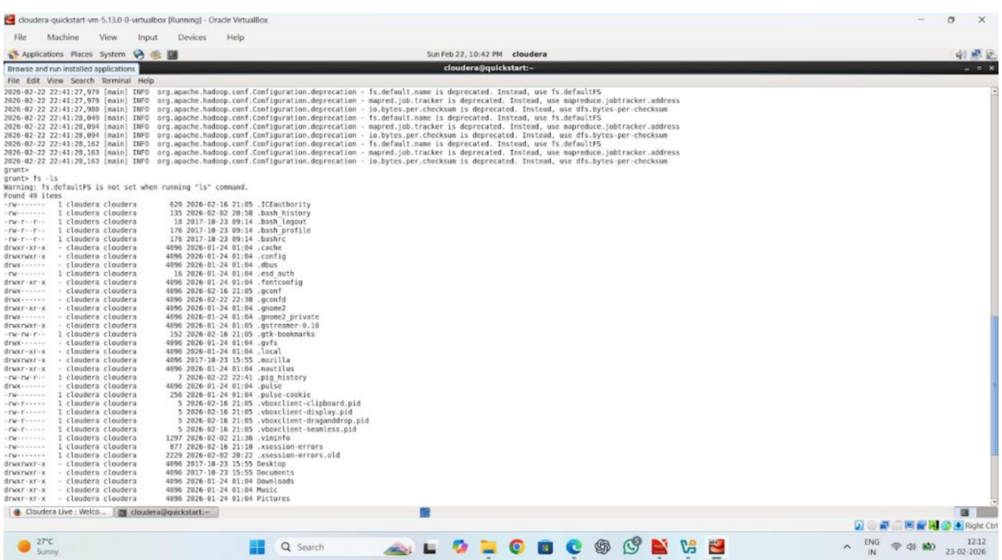
[cloudera@quickstart ~]$ pig -x local



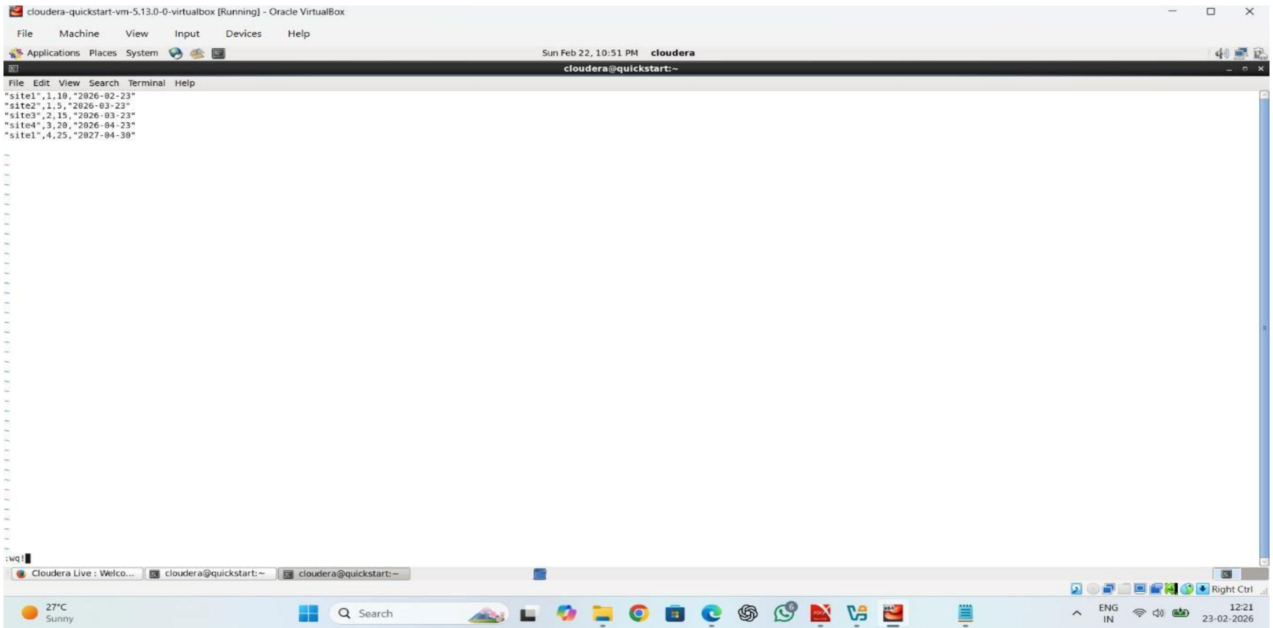
grunt>

--command to see file list

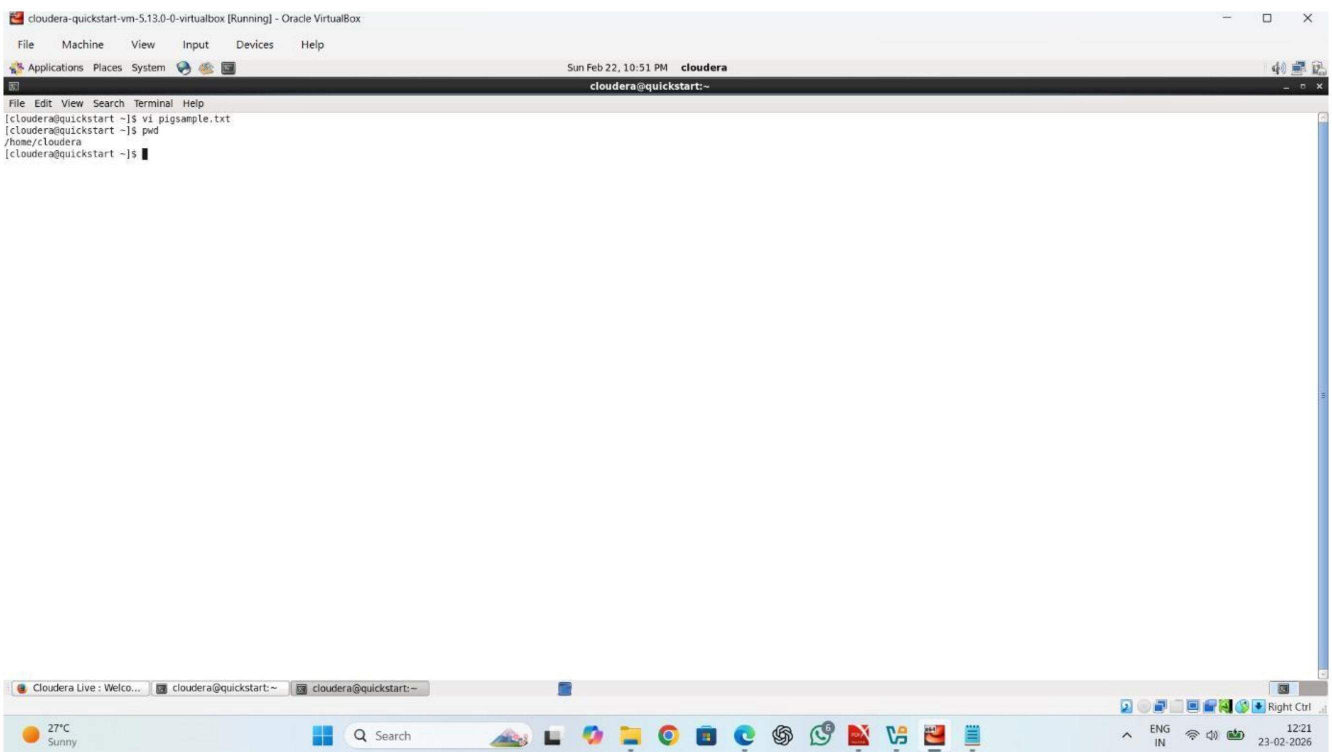
grunt> fs -ls



--load file



cloudera-quickstart-vm-5.13.0-0-virtualbox [Running] - Oracle VirtualBox
cloudera@quickstart:~
"site1",1,10,"2026-02-23"
"site2",1,5,"2026-03-23"
"site3",2,15,"2026-03-23"
"site4",3,20,"2026-04-23"
"site1",4,25,"2027-04-30"
:wq!

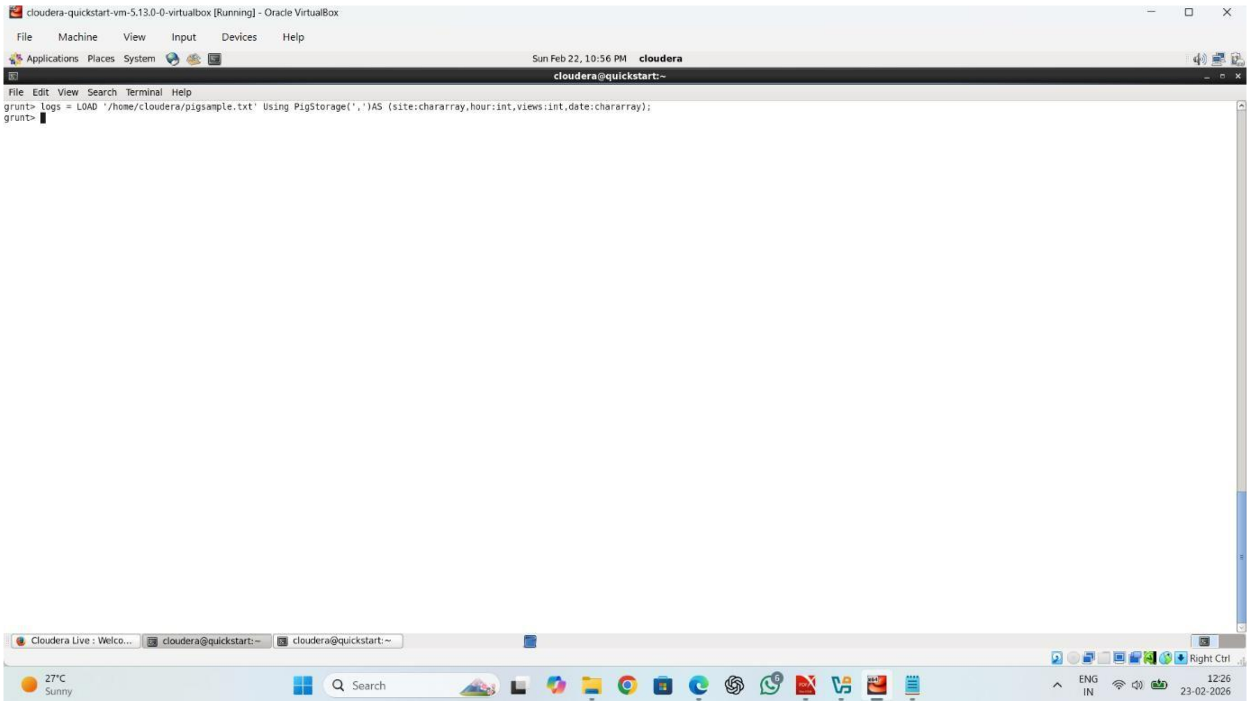



cloudera-quickstart-vm-5.13.0-0-virtualbox [Running] - Oracle VirtualBox
cloudera@quickstart:~
[cloudera@quickstart ~]$ vi pigsample.txt
[cloudera@quickstart ~]$ pwd
/home/cloudera
[cloudera@quickstart ~]$

**VNRVJIET** **Name of the Experiment:** ____________

**Name of the Laboratory:** ____________

**Experiment No:**______ **Date:** ____________




```
A = LOAD '/home/cloudera/java.txt' USING PigStorage(',') AS
(a1:int,a2:chararray,a3:chararray,a4:chararray) ;
--execute and verify data
dump A;
```

------------------------

```
-- get storage structure
describe A ;
```

-----------------------------

```
---cross join
A = LOAD '/home/cloudera/M.txt' USING PigStorage(',') AS (a1:int,a2:int)
; dump A;
B = LOAD '/home/cloudera/N.txt' USING PigStorage(',') AS (a1:int,a2:int) ;

dump B;
```

```
result = cross A,B;
```

**-- distinct**

```
A = LOAD '/home/cloudera/M.txt' USING PigStorage(',') AS (a1:int,a2:int)
; dump A;
Result = DISTINCT A;
dump Result
```

**----filter command**

```
A = LOAD '/home/cloudera/java.txt' USING PigStorage(',') AS
(a1:int,a2:chararray,a3:chararray,a4:chararray) ;
dump A
result = filter A by a1>9
dump Result
```

**---Foreach command**

```
A = LOAD '/home/cloudera/java.txt' USING PigStorage(',') AS
(a1:int,a2:chararray,a3:chararray,a4:chararray) ;
dump A
result= foreach A generate a1*10,a2;
dump Result;
```

**---group command**

```
A = LOAD '/home/cloudera/alldata.txt' USING PigStorage(',') AS
(a1:int,a2:chararray,a3:chararray,a4:chararray) ;
dump A
```

result = group A by a4 ;

dump result;

**-cogroup command**

A = LOAD '/home/cloudera/java.txt' USING PigStorage(',') AS (a1:int,a2:chararray,a3:chararray,a4:chararray) ;

dump A

B = LOAD '/home/cloudera/Hadoop' USING PigStorage(',') AS (a1:int,a2:chararray,a3:chararray,a4:chararray) ;

dump B

CG= cogroup A by a4, B by a4

**---Limit command -- limit no of rows**
result = LIMIT B

2; dump result;

- - - - - - - - - - - - - - - - - - - - - - - - - -

---order/ sorting

command result = order

B by a4 desc dump

result;

- - - - - - - - - - - - - - - - - - - - - - - - -

**--split command**

SPLIT A INTO X IF a1<=2, Y IF a1>2;

dum

p X;

dum

p Y;

-----------------

--union

command Z =

union X,Y

dump Z

------------------

empcode int

empfname string

emplname

string

job string

manager string

hiredate string

salary int

commission int

deptcode int

----------------------------------

**---Aggrigate functions:**

A = LOAD '/home/cloudera/employee_data.txt' USING PigStorage(',') AS (empcode:int,empfname:chararray,emplname:chararray,job:chararray, manager:chararray, hiredate:chararray,salary:int, commission:int, deptcode:int);

```
dump A;
B = group A by
job; dump B;
C= foreach B generate group,
SUM(A.salary); dump C;
```

- - - - - - - - - - - - - - - - - - - - - - - - - - -

```
--string function concat
A = LOAD '/home/cloudera/employee_data.txt' USING PigStorage(',') AS
(empcode:int,empfname:chararray,emplname:chararray,job:chararray,
manager:chararray, hiredate:chararray,salary:int, commission:int, deptcode:int);
dump A;
C= foreach A generate CONCAT(A.empfname,A.emplname),
A.salary; dump C;

--mathamatical function
A = LOAD '/home/cloudera/employee_data.txt' USING PigStorage(',') AS
(empcode:int,empfname:chararray,emplname:chararray,job:chararray,
manager:chararray, hiredate:chararray,salary:int, commission:int, deptcode:int);
dump A;
B = group A by
job; dump B;
C= foreach B generate A.job,
FLOOR(AVG(A.salary)); dump C;
C= foreach B generate A.job, FLOOR(AVG(A.salary));
```

**---self join**

A = LOAD '/home/cloudera/employee_data.txt' USING PigStorage(',') AS (empcode:int,empfname:chararray,emplname:chararray,job:chararray, manager:chararray, hiredate:chararray,salary:int, commission:int, deptcode:int);

dump A;

B = LOAD '/home/cloudera/dept_data.txt' USING PigStorage(',') AS (deptcode:int, deptname:chararray, location:chararray);

dump B;

dept3 = JOIN A BY deptcode, B BY

deptcode; dump dept3

dept3 = JOIN A BY deptcode, B BY

deptcode; dump dept 3

dept3 = JOIN A BY deptcode LEFT OUTER, B BY

deptcode; dump dept 3

dept3 = JOIN A BY deptcode RIGHT OUTER, B BY

deptcode; dump dept 3

dept3 = JOIN A BY deptcode FULL OUTER, B BY

deptcode; dump dept 3

**---- Sum of two numbers;**

A = LOAD '/home/cloudera/employee_data.txt' USING PigStorage(',') AS (empcode:int,empfname:chararray,emplname:chararray,job:chararray, manager:chararray, hiredate:chararray,salary:int, commission:int, deptcode:int);

dump A;

B= foreach A generate salary+commission as TotalSal,salary,commission

C= foreach A generate empcode, salary, empfname, emplname, salary+empcode

- - - - - - - - - - - - - - - - - - - - - - - - -

---- How to run .pig file

grunt> exec /home/cloudera/wordcount.pig

- - - - - - - - - - - - - - - - - - - -

**word count**

```
nput2 = LOAD '/home/cloudera/wordcount.txt' AS (line:Chararray);

 Words = FOREACH input2 GENERATE FLATTEN(TOKENIZE(line,' '))
AS word;

Grouped = GROUP Words BY word;
wordcount = FOREACH Grouped GENERATE COUNT(Words),group;
dump wordcount;
```

- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -

**—–department wise maximum salary**

```
A = LOAD '/home/cloudera/employee_data.txt' USING PigStorage(',') AS
(empcode:int,empfname:chararray,emplname:chararray,job:chararray,
 manager:chararray, hiredate:chararray,salary:int, commission:int, deptcode:int);
dump A;
B = group A by deptcode
dump B;
C = foreach B generate group, MAX(salary);
dump C;
```

```
A = LOAD '/home/cloudera/employee_data.txt' USING PigStorage(',') AS
(empcode:int,empfname:chararray,emplname:chararray,job:chararray,
manager:chararray, hiredate:chararray,salary:int, commission:int, deptcode:int);

dump A;

B = order A by job;

C = foreach B generate empcode, concat(empfname+emplname), substr(job,1,3)

dump C;

C = foreach A generate CONCAT(empfname,emplname),
SUBSTRING(job,1,3),empcode,hiredate,salary;
```

**Experiment 5**

4.SQOOP

I. Create table in HIVE using hive query
language. [cloudera@quickstart ~]$ hive
hive> show
databases;
O/P:-
OK
hive> use
default;
O/P:-
OK
Aks
Aksra
ara
default

Time taken: 0.026 seconds
hive> show tables;
O/P:- OK
my_table
user_data
user_data1

hive> CREATE TABLE my_table (

> id INT,
> name STRING,
> age INT
> )
> ROW FORMAT DELIMITED

```
  > FIELDS TERMINATED BY ','
  > STORED AS
    TEXTFILE;
> O/P:- OK
hive> INSERT INTO my_table VALUES
    > (1, 'Alice', 25),
    > (2, 'Bob', 30),
    > (3, 'Charlie', 35),
    > (4, 'Dave', 40),
    > (5, 'Eve', 45),
    > (6, 'Frank', 50),
    > (7, 'Grace', 55),
    > (8, 'Henry', 60),
    > (9, 'Isabel', 65),
    > (10,'John', 70);

  > O/P:-
Loading data to table default.my_table
Table default.my_table stats: [numFiles=1, numRows=10, totalSize=108,
  rawDataSize=98]
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1     Cumulative CPU: 1.08 sec   HDFS Read:
  4115 HDFS
Write: 180 SUCCESS
Total MapReduce CPU Time Spent: 1 seconds 80 msec
  OK
```

II. Import the sql table data into hive using sqoop too.

To activate MySQL : $ mysql -u root -pcloudera MySQL Database name : retail_db

MySQL Table name : categories

```
$ sqoop import \
```

--connect jdbc:mysql://localhost/retail_db \
--username root \
--password cloudera \
--table categories \
--hive-import \
--hive-table demo3

III. Export hive table data into local machine and into SQL.

1) Create a hive table with data (categories)

   a) Use DESCRIBE FORMATTED TABLE_NAME; to check the location and field delimiters. ex: '/0001',',' etc.

2) Create a MySQL table (cats) with same schema as the hive table

3) Run the following command in cloudera and check in MySQL

$ sqoop export --connect jdbc:mysql://localhost/mysql -m 1 --table cats –

export-dir /user/hive/warehouse/categories --input-fields-terminated-by

'\0001' --username root --password cloudera;

**Experiment 6**

Time series: when all other factors are constant prediction of future values.

• Different Methods of doing Time series Analysis and Forecasting

o ARIMA model.

o Seasonally ARIMA. Most used.

o Holt Winter Exponential Smoothing. Easiest and effective model. Link

o Advanced Models

1. **Important Concepts and terminology in Time series Analysis.**

o Stationarity. To know everything follow the link.

▪ A stationary time series is one whose properties (ie mean, variance, autocorrelation) does not depend on the time.

o Autoregression AR.

o Moving Average MA

o Integration & Difference

o ACF and PACF Plots

o Time series components:

▪ Trend: long term smooth movement, upward or downward

▪ Seasonal: periodic fluctuation, less than 1 year, most commonly found in industry.

▪ Cyclical: periodic fluctuation, more than 1 year.

▪ irregularity: random movement.

ARIMA is the Most common model used for time series forecasting. It has 3 components.

1. **Autoregression AR.**
2. **Moving Average MA**
3. **Integrated**

1. **Autoregression AR.**

o Future values of Y is dependent of previous lagged values of Y.

o regression of yt on yt-1, yt-2 .

o P = ORDER OF AR; current value of y is dependent on how many previous lagged value of current Y. if p=2 that means yt is dependent on yt-1 and yt-2.

o P from PACF o Interpretation of PACF:

2. **Moving Average MA.**

Future values of Y is dependent of previous lagged values of white noise ie the irregular component. white noise is just the error. error is the difference between the actual value and predicted value. so we take into consideration the error also to predict the future value.

o autocorrelation between the errors.

o Trend, s, c components of TS is captured in AR whereas the irregular comp is captured in MA.

o q is order of MA.

o ACF gives q.

3. **Integrated**

o Integrated means no of times we difference the data then we have to integrated it back to get original series back.

o We difference to remove trend and seasonality to it stationary series as only after making a series stationary we can implement AR and MA.

# ARIMA and Seasonal ARIMA

## Autoregressive Integrated Moving Averages

The general process for ARIMA models is the following:

- Visualize the Time Series Data
- Make the time series data stationary
- Plot the Correlation and AutoCorrelation Charts
- Construct the ARIMA Model or Seasonal ARIMA based on the data
- Use the model to make predictions Let's go through these steps!

In [3]:
```
import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
%matplotlib inline
```

In [4]:
```
df=pd.read_csv('perrin-freres-monthly-champagne-.csv')
```

In [5]:
```
df.head()
```

Out[5]:

| | Month | Perrin Freres monthly champagne sales millions ?64-?72 |
|---|---|---|
| **0** | 1964-01 | 2815.0 |
| **1** | 1964-02 | 2672.0 |
| **2** | 1964-03 | 2755.0 |
| **3** | 1964-04 | 2721.0 |
| **4** | 1964-05 | 2946.0 |

In [6]:
```
df.tail()
```

Out[6]:

| | Month | Perrin Freres monthly champagne sales millions ?64-?72 |
|---|---|---|
| **102** | 1972-07 | 4298.0 |
| **103** | 1972-08 | 1413.0 |

In [7]:
```
## Cleaning up the data
df.columns=["Month","Sales"]
df.head()
```

Out[7]:

| | Month | Sales |
|---|---|---|
| 0 | 1964-01 | 2815.0 |
| 1 | 1964-02 | 2672.0 |
| 2 | 1964-03 | 2755.0 |
| 3 | 1964-04 | 2721.0 |
| 4 | 1964-05 | 2946.0 |

In [9]:

In [8]:
```
## Drop last 2 rows
df.drop(106,axis=0,inplace=True)
```

```
df.tail()
```

Out[9]:

| | Month | Sales |
|---|---|---|
| 101 | 1972-06 | 5312.0 |
| 102 | 1972-07 | 4298.0 |
| 103 | 1972-08 | 1413.0 |
| 104 | 1972-09 | 5877.0 |
| 105 | NaN | NaN |

In [11]:

In [10]:
```
df.drop(105,axis=0,inplace=True)
```

```
df.tail()
```

Out[11]:

| | Month | Sales |
|---|---|---|
| 100 | 1972-05 | 4618.0 |
| 101 | 1972-06 | 5312.0 |
| 102 | 1972-07 | 4298.0 |
| 103 | 1972-08 | 1413.0 |
| 104 | 1972-09 | 5877.0 |

In [12]:
```
# Convert Month into Datetime
df['Month']=pd.to_datetime(df['Month'])
```

| | |
|---|---|
| **0** | 1964-01-012815.0 |
| **1** | 1964-02-012672.0 |
| **2** | 1964-03-012755.0 |
| **3** | 1964-04-012721.0 |
| **4** | 1964-05-012946.0 |

In [15]:

In [14]:
```
df.set_index('Month',inplace=True)
```

```
df.head()
```

Out[15]:

| | **Sales** |
|---|---|
| **Month** | |
| **1964-01-01** | 2815.0 |
| **1964-02-01** | 2672.0 |
| **1964-03-01** | 2755.0 |
| **1964-04-01** | 2721.0 |
| **1964-05-01** | 2946.0 |

In [16]:

```
df.describe()
```

Out[16]:

| | **Sales** |
|---|---|
| **count** | 105.000000 |
| **mean** | 4761.152381 |
| **std** | 2553.502601 |
| **min** | 1413.000000 |
| **25%** | 3113.000000 |
| **50%** | 4217.000000 |
| **75%** | 5221.000000 |
| **max** | 13916.000000 |

## Step 2: Visualize the Data

# VNRVJIET

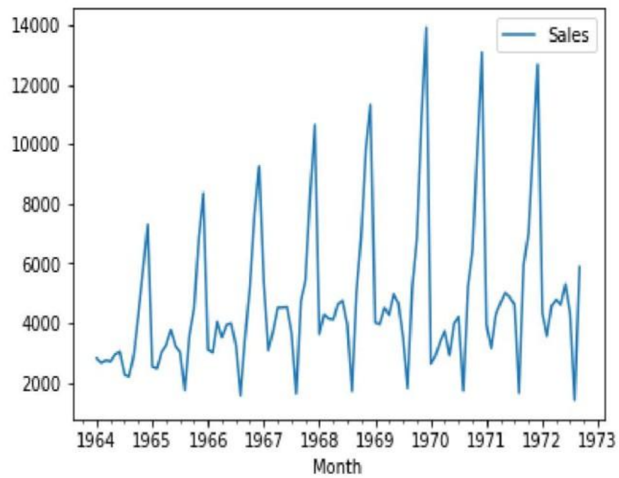
**Name of the Experiment:** ______________

**Name of the Laboratory:** ______________

**Experiment No:**_______ **Date:** ______________

In [17]:
```
df.plot()
```

Out[17]: <matplotlib.axes._subplots.AxesSubplot at 0x1d2c881a2e8>




In [18]:
```
### Testing For Stationarity

from statsmodels.tsa.stattools import adfuller
```

In [19]:
```
test_result=adfuller(df['Sales'])
```

In [20]:
```
#Ho: It is non stationary
#H1: It is stationary

def adfuller_test(sales):
    result=adfuller(sales)
    labels = ['ADF Test Statistic','p-value','#Lags Used','Number of Observati
    for value,label in zip(result,labels):
        print(label+' : '+str(value) )
    if result[1] <= 0.05:
        print("strong evidence against the null hypothesis(Ho), reject the nul
    else:
        print("weak evidence against null hypothesis, time series has a unit r
```

In [21]:
```
adfuller_test(df['Sales'])
```

```
ADF Test Statistic : -1.8335930563276297
p-value : 0.3639157716602417
#Lags Used : 11
Number of Observations Used : 93
weak evidence against null hypothesis, time series has a unit root, indicatin
g it is non-stationary
```

## Differencing

```
In [22]: df['Sales First Difference'] = df['Sales'] - df['Sales'].sift(1)
In [212]: df['Sales'].shift(1)
```

Out[212]:

```
Month
1964-01-01       NaN
1964-02-01    2815.0
1964-03-01    2672.0
1964-04-01    2755.0
1964-05-01    2721.0
1964-06-01    2946.0
1964-07-01    3036.0
1964-08-01    2282.0
1964-09-01    2212.0
1964-10-01    2922.0
1964-11-01    4301.0
1964-12-01    5764.0
1965-01-01    7312.0
1965-02-01    2541.0
1965-03-01    2475.0
1965-04-01    3031.0
1965-05-01    3266.0
1965-06-01    3776.0
1965-07-01    3230.0
1965-08-01    3028.0
1965-09-01    1759.0
1965-10-01    3595.0
1965-11-01    4474.0
1965-12-01    6838.0
1966-01-01    8357.0
1966-02-01    3113.0
1966-03-01    3006.0
1966-04-01    4047.0
1966-05-01    3523.0
1966-06-01    3937.0
                 ...
1970-04-01    3370.0
1970-05-01    3740.0
1970-06-01    2927.0
1970-07-01    3986.0
1970-08-01    4217.0
1970-09-01    1738.0
1970-10-01    5221.0
1970-11-01    6424.0
1970-12-01    9842.0
```

```
1971-07-01     4874.0
1971-08-01     4633.0
1971-09-01     1659.0
1971-10-01     5951.0
1971-11-01     6981.0
1971-12-01     9851.0
1972-01-01    12670.0
1972-02-01     4348.0
1972-03-01     3564.0
1972-04-01     4577.0
1972-05-01     4788.0
1972-06-01     4618.0
1972-07-01     5312.0
1972-08-01     4298.0
1972-09-01     1413.0
Name: Sales, Length: 105, dtype: float64
```

khjbdcbn

In [190]:

In [188]:
```python
df['Seasonal First Difference']=df['Sales']-df['Sales'].shift(12)
```

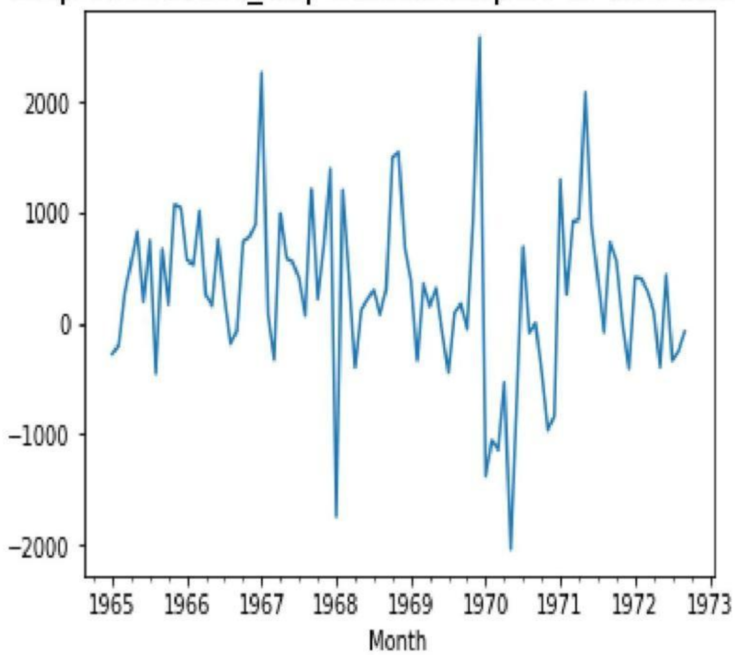
```python
df.head(14)
```

Out[190]:

| | Sales | Sales First Difference | forecast | Seasonal First Difference | Month |
|---|---|---|---|---|---|
| **1964-01-01** | 2815.0 | NaN | NaN | NaN | |
| **1964-02-01** | 2672.0 | -143.0 | NaN | NaN | |
| **1964-03-01** | 2755.0 | 83.0 | NaN | NaN | |
| **1964-04-01** | 2721.0 | -34.0 | NaN | NaN | |
| **1964-05-01** | 2946.0 | 225.0 | NaN | NaN | |
| **1964-06-01** | 3036.0 | 90.0 | NaN | NaN | |
| **1964-07-01** | 2282.0 | -754.0 | NaN | NaN | |
| **1964-08-01** | 2212.0 | -70.0 | NaN | NaN | |
| **1964-09-01** | 2922.0 | 710.0 | NaN | NaN | |
| **1964-10-01** | 4301.0 | 1379.0 | NaN | NaN | |
| **1964-11-01** | 5764.0 | 1463.0 | NaN | NaN | |
| **1964-12-01** | 7312.0 | 1548.0 | NaN | NaN | |
| **1965-01-01** | 2541.0 | -4771.0 | NaN | -274.0 | |
| **1965-02-01** | 2475.0 | -66.0 | NaN | -197.0 | |

In [193]:
```
df['Seasonal First Difference'].plot()
```

Out[193]: <matplotlib.axes._subplots.AxesSubplot at 0x1d2da817e80>




## Auto Regressive Model

$$y_t = c + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \cdots + \phi_p y_{t-p} + \varepsilon_t,$$

In [54]:
```
from pandas.tools.plotting import autocorrelation_plot
autocorrelation_plot(df['Sales'])
plt.show()
```

```
C:\Users\krish.naik\AppData\Local\Continuum\anaconda3\lib\site-
packages\ipyke rnel_launcher.py:2: FutureWarning:
'pandas.tools.plotting.autocorrelation_plo t' is deprecated, import
'pandas.plotting.autocorrelation_plot' instead.
```

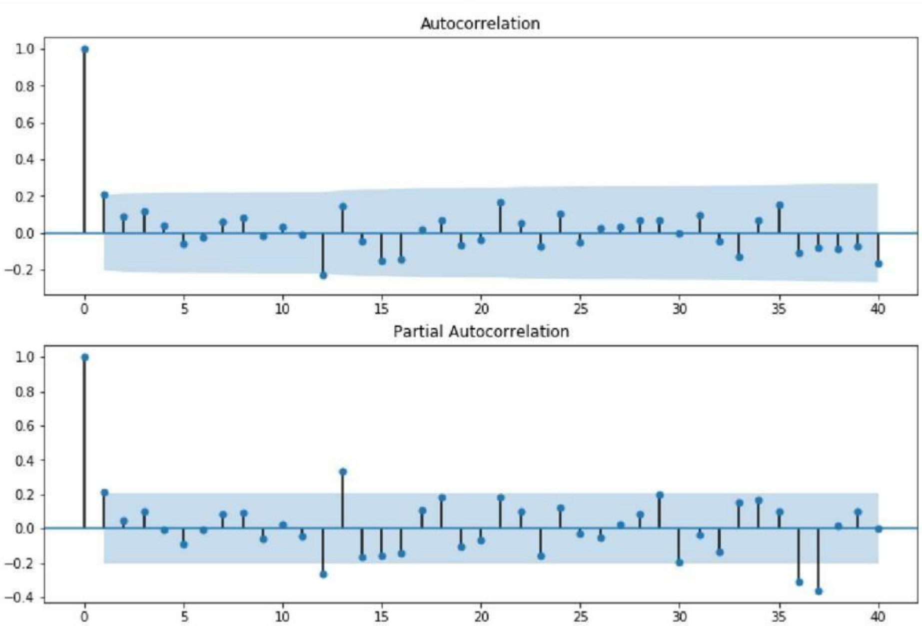
- For an AR model, the theoretical PACF "shuts off" past the order of the model. The phrase "shuts off" means that in theory the partial autocorrelations are equal to 0 beyond that point. Put another way, the number of non-zero partial autocorrelations gives the order of the AR model. By the "order of the model" we mean the most extreme lag of x that is used as a predictor.

- Identification of an MA model is often best done with the ACF rather than the PACF.
  - For an MA model, the theoretical PACF does not shut off, but instead tapers toward 0 in some manner. A clearer pattern for an MA model is in the ACF. The ACF will have non-zero autocorrelations only at lags involved in the model.

  p,d,q p AR model lags d differencing q MA lags

```
In [26]: from statsmodels.graphics.tsaplots import plot_acf,plot_pacf
```

```
In [203]: fig = plt.figure(figsize=(12,8))
          ax1 = fig.add_subplot(211)
          fig = sm.graphics.tsa.plot_acf(df['Seasonal First Difference'].iloc[13:],lags=
          ax2 = fig.add_subplot(212)
          fig = sm.graphics.tsa.plot_pacf(df['Seasonal First Difference'].iloc[13:],lags
```




```
In [115]: # For non-seasonal data
          #p=1, d=1, q=0 or 1
          from statsmodels.tsa.arima_model import ARIMA
In [176]: model=ARIMA(df['Sales'],order=(1,1,1))
          model_fit=model.fit()
```

```
C:\Users\krish.naik\AppData\Local\Continuum\anaconda3\lib\site-packages\stats
models\tsa\base\tsa_model.py:171: ValueWarning: No frequency information was
provided, so inferred frequency MS will be used.
  % freq, ValueWarning)
C:\Users\krish.naik\AppData\Local\Continuum\anaconda3\lib\site-packages\stats
models\tsa\base\tsa_model.py:171: ValueWarning: No frequency information was
provided, so inferred frequency MS will be used.   % freq, ValueWarning)
```

In [177]:
```
model_fit.summary()
```

Out[177]:

ARIMA Model Results

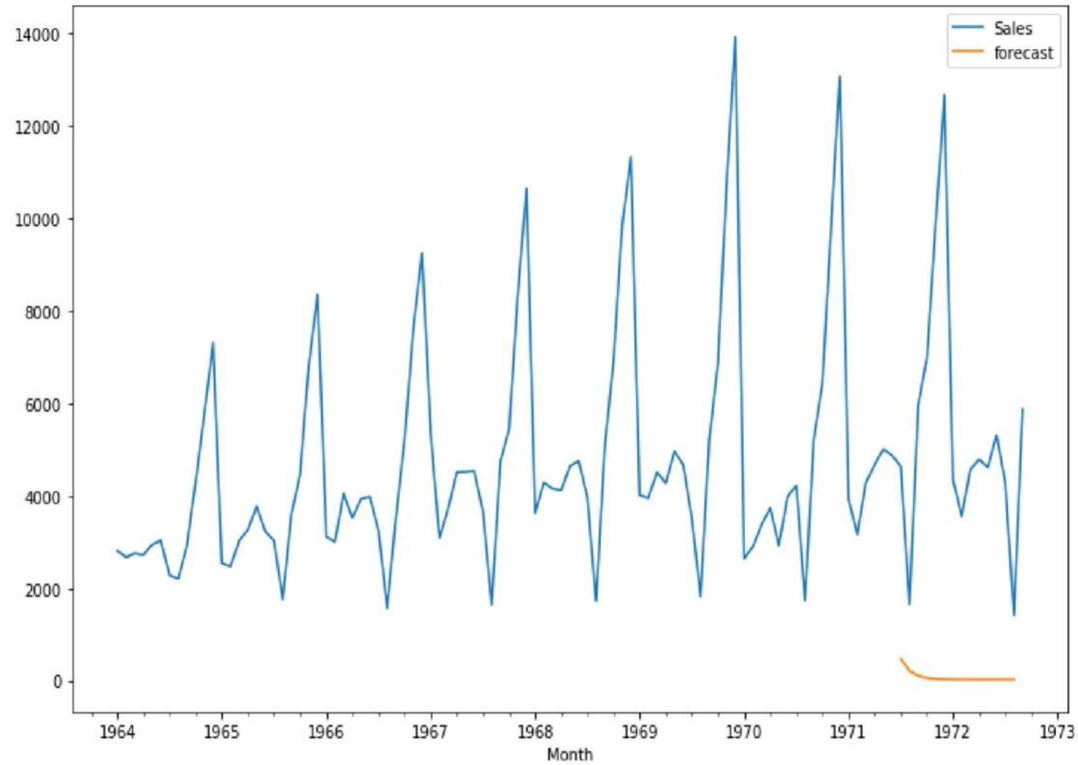
| | | | |
|---|---|---|---|
| **Dep. Variable:** | D.Sales | **No. Observations:** | 104 |
| **Model:** | ARIMA(1, 1, 1) | **Log Likelihood** | -951.126 |
| **Method:** | css-mle | **S.D. of innovations** | 2227.262 |
| **Date:** | Wed, 18 Mar 2020 | **AIC** | 1910.251 |
| **Time:** | 13:40:32 | **BIC** | 1920.829 |
| **Sample:** | 02-01-1964 | **HQIC** | 1914.536 |
| | - 09-01-1972 | | |

| | **coef** | **std err** | **z** | **P>\|z\|** | **[0.025** | **0.975]** |
|---|---|---|---|---|---|---|
| **const** | 22.7838 | 12.405 | 1.837 | 0.069 | -1.530 | 47.098 |
| **ar.L1.D.Sales** | 0.4343 | 0.089 | 4.866 | 0.000 | 0.259 | 0.609 |
| **ma.L1.D.Sales** | -1.0000 | 0.026 | -38.503 | 0.000 | -1.051 | -0.949 |

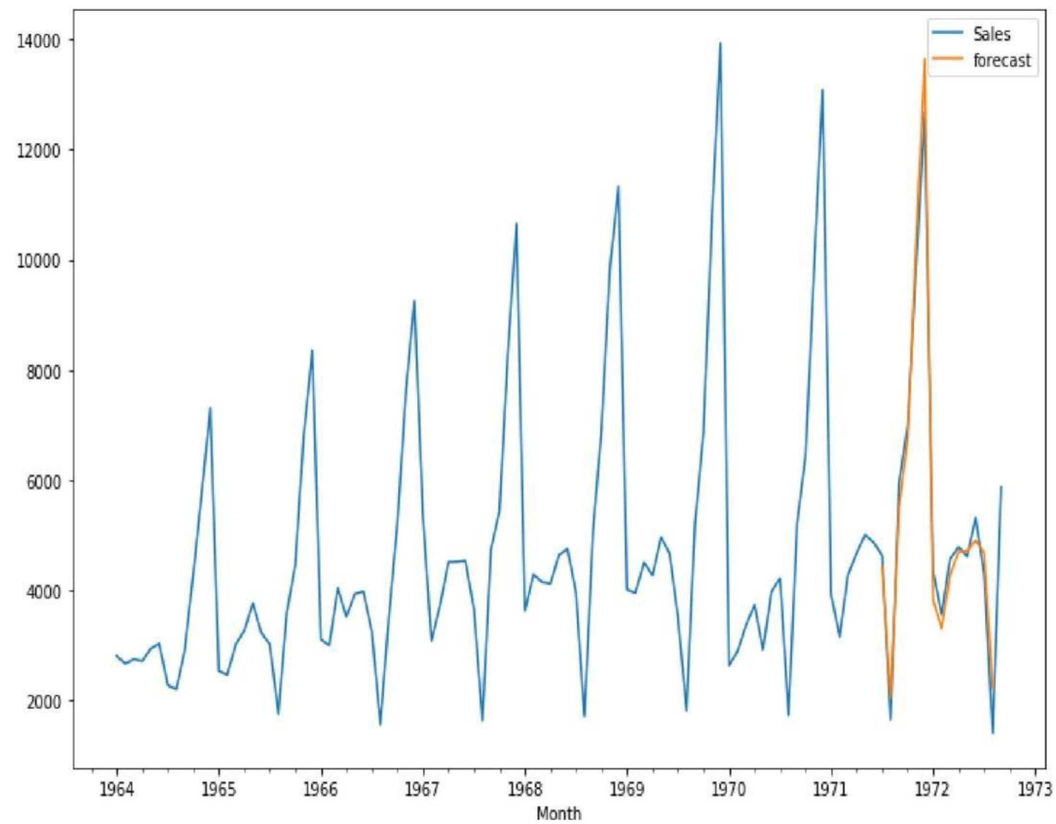
Roots

| | **Real** | **Imaginary** | **Modulus** | **Frequency** |
|---|---|---|---|---|
| **AR.1** | 2.3023+0.0000j | | 2.3023 | 0.0000 |
| **MA.1** | 1.0000+0.0000j | | 1.0000 | 0.0000 |

```
In [178]: df['forecast']=model_fit.predict(start=90,end=103,dynamic=True)
          df[['Sales','forecast']].plot(figsize=(12,8))

Out[178]: <matplotlib.axes._subplots.AxesSubplot at 0x1d2d9f1fda0>
```




```
In [113]: import statsmodels.api as sm

In [204]: model=sm.tsa.statespace.SARIMAX(df['Sales'],order=(1, 1, 1),seasonal_order=(1,
          results=model.fit()

          C:\Users\krish.naik\AppData\Local\Continuum\anaconda3\lib\site-
          packages\stats models\tsa\base\tsa_model.py:171: ValueWarning: No
          frequency information was provided, so inferred frequency MS will be used.
          % freq, ValueWarning)

In [205]: df['forecast']=results.predict(start=90,end=103,dynamic=True)
          df[['Sales','forecast']].plot(figsize=(12,8))

Out[205]: <matplotlib.axes._subplots.AxesSubplot at 0x1d2db70e278>

In [209]: future_df=pd.concat([df,future_datest_df])
```




In [206]:
```
from pandas.tseries.offsets import DateOffset
future_dates=[df.index[-1]+ DateOffset(months=x)for x in range(0,24)]
```

In [207]:
```
future_datest_df=pd.DataFrame(index=future_dates[1:],columns=df.columns)
```

In [208]:
```
future_datest_df.tail()
```

Out[208]:

| | Sales | Sales First Difference | forecast | Seasonal First Difference |
|---|---|---|---|---|
| **1974-04-01** | NaN | NaN | NaN | NaN |
| **1974-05-01** | NaN | NaN | NaN | NaN |
| **1974-06-01** | NaN | NaN | NaN | NaN |
| **1974-07-01** | NaN | NaN | NaN | NaN |
| **1974-08-01** | NaN | NaN | NaN | NaN |

In

```
[201]: future_df['forecast'] = results.predict(start = 104, end = 120,
dynamic= True) future_df[['Sales', 'forecast']].plot(figsize=(12, 8))
```

Out[201]: <matplotlib.axes._subplots.AxesSubplot at 0x1d2daee5048>